

STUDY GUIDE · FROM ZERO TO RUNNING IT YOURSELF

SLO-Driven Progressive Delivery with Automated Rollback

Every tool explained for beginners: Docker, Kubernetes, kind, Helm, SLOs and error budgets, Prometheus, Argo Rollouts, Grafana, GitHub Actions and security tooling, plus 8 hands-on labs, a glossary and 25 interview questions.

Sufyan Ahmad · DevOps Engineer

<https://github.com/sufyanahmadkamboh/sufyan-devops-slo-progressive-delivery>

Contents

- 0. The big picture (no jargon)
- 1. Docker and containers
- 2. Kubernetes
- 3. kind (Kubernetes in Docker)
- 4. Helm
- 5. SRE basics: SLI, SLO, error budget, burn rate
- 6. Prometheus and PromQL
- 7. Progressive delivery and Argo Rollouts
- 8. Grafana
- 9. GitHub Actions (CI/CD)
- 10. Quality and security tools
- 11. How everything fits together
- 12. Hands-on labs

Glossary

Interview questions this project prepares you for

Each chapter: what the tool is, why this project uses it, how it works, where it is integrated, commands to try, common mistakes, and questions with answers.

0. The big picture (no jargon)

The problem, as a story

A team runs an online shop. Its "orders" service answers when a customer opens *My orders*. Every week developers ship a new version.

The usual way to release: replace the old version with the new one for everyone. If the new version has a bug, *every* customer hits it at once. Someone gets an alarm at night, has to decide "is this bad enough to undo?", and undoes it by hand.

What this project does instead: it releases the new version like a careful restaurant tests a new dish. 1. Serve it to **1 table out of 5** first, and **watch** what happens. 2. If the guests are happy, serve it to 2 tables, then 3, then 4, then everyone. 3. If guests complain, **take it off the menu immediately**, and everyone keeps getting the old dish.

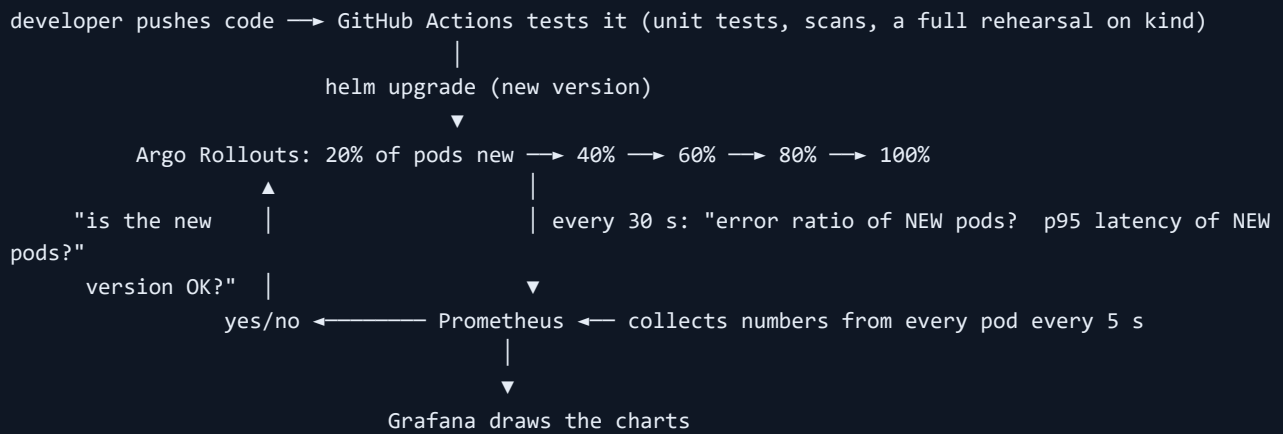
"Watching" is done with numbers, not opinions: - How many requests to the new version **fail**? - How **slow** is the new version?

Before any release, the team agreed on the limits: "at most 7.2% failures" and "95% of requests faster than 300 ms". A computer checks these numbers every 30 seconds and makes the decision, so nobody has to wake up.

The pieces, and their jobs

Piece	Job in one sentence	Restaurant analogy
orders-api (our app)	Answers requests and counts how many succeed or fail	The kitchen
Docker	Packs the app with everything it needs into one portable box	Meal-prep boxes that taste the same anywhere
Kubernetes	Runs many copies of the boxes and keeps them alive	The restaurant manager who assigns cooks and replaces sick ones
kind	A small but real Kubernetes running on your laptop	A practice kitchen at home
Helm	Installs all the Kubernetes settings in one command, with versions	The recipe book
Prometheus	Collects the numbers (requests, errors, speed) every 5 seconds	The waiter writing down every complaint and how long each dish took
Argo Rollouts	Shifts traffic step by step and decides "continue or undo" from the numbers	The head chef who decides whether the new dish stays
Grafana	Draws the numbers as charts	The whiteboard in the kitchen
GitHub Actions	Tests everything automatically on every code change	The food inspector who checks before the restaurant opens

The flow, in one picture



What you will be able to do after this guide

- Explain each tool and **why it was chosen**.
- Run the system, release versions, and watch a bad release get rolled back on its own.
- Read and change the Kubernetes, Helm, Prometheus and CI files.
- Answer the interview questions in [interview-questions.md](#).

1. Docker and containers

What is it?

The problem Docker solves: a program needs more than its own code to run. It needs a language runtime (here Python 3.13), libraries (here `prometheus-client`) and an operating system underneath. "It works on my machine" happens when those differ between computers.

A **container image** is a snapshot that contains the program **and** everything it needs. A **container** is a running copy of that image. **Docker** is the tool that builds images and runs containers.

Term	Meaning
Dockerfile	The recipe: step-by-step instructions to build the image
Image	The built result: read-only, versioned with a tag (<code>orders-api:1.1.0</code>)
Container	A running instance of an image, isolated from other processes
Registry	A storage place for images (Docker Hub, GHCR, ECR). This lab loads images straight into kind instead

A container is not a virtual machine. It shares the host's kernel, so it starts in milliseconds and uses little memory.

Why this project uses it

- **Kubernetes only runs containers**, so the app must be packaged as an image.
- **Each release is a new image tag** (`1.1.0` , `1.2.0` , ...). Argo Rollouts compares old and new versions, and that only works if each version is an immutable, separately tagged artifact.

How it works in this project

The recipe is `app/Dockerfile` . Line by line:

```
FROM python:3.13-slim AS build          # stage 1: start from an official Python image
COPY requirements.txt .
RUN pip install --prefix=/install -r requirements.txt  # install libraries into /install

FROM python:3.13-slim                  # stage 2: start clean again (the "runtime" image)
RUN apt-get update && apt-get upgrade -y ... \
  && python -m pip uninstall -y setuptools wheel pip \  # remove tools not needed at runtime
  && useradd --uid 10001 ... app                # create a non-root user
COPY --from=build /install /usr/local        # copy ONLY the installed libraries from stage 1
COPY orders_api/ orders_api/                # copy our code
USER 10001                                   # never run as root
ENTRYPOINT ["python", "-m"]
CMD ["orders_api.server"]                   # default program; the load generator overrides this
```

Key ideas used here:

Idea	Where	Why it matters
Multi-stage build	two <code>FROM</code> lines	Build tools stay in stage 1 and the final image is smaller with fewer vulnerabilities
Non-root user	<code>USER 10001</code>	If the app is hacked, the attacker isn't root inside the container
Remove pip	<code>pip uninstall ...</code> <code>pip</code>	The security scan (Trivy) found vulnerabilities inside pip's bundled libraries. The app doesn't need pip at runtime, so it is removed
Build argument	<code>ARG APP_VERSION</code>	The version number is baked into the image, and the app reports it at <code>/version</code>
.dockerignore	<code>app/.dockerignore</code>	Keeps tests and caches out of the image

The same image is used two ways: - the API: `CMD orders_api.server` - the load generator: the Helm chart passes `args: ["orders_api.loadgen"]`

One artifact serving two roles is simpler to maintain.

Where images are built: `scripts/build-image.sh` runs `docker build` and then `kind load docker-image`. That copies the image straight into the kind cluster's nodes, so no registry is needed for the lab.

Try it

```

docker build -t orders-api:study app                # build the image
docker image ls orders-api                          # see its size and tag
docker run --rm -p 8080:8080 orders-api:study        # run it (Ctrl+C to stop)
# in another terminal:
curl localhost:8080/api/orders
curl localhost:8080/metrics | head -20              # the numbers Prometheus will collect
docker run --rm --entrypoint id orders-api:study     # proves it runs as uid 10001, not root
docker run --rm -e FAULT_ERROR_RATE=0.5 -p 8080:8080 orders-api:study # now about half the requests fail

```

Common mistakes

- **Running as root:** this is the default in many images. Always add a `USER`.
- **One huge single-stage image** that carries compilers and caches into production.
- **The `latest` tag in production:** you can no longer tell which version is running, or roll back to "the previous one".

Check yourself

1. Why are there two `FROM` lines?
2. What would break if every release used the same tag, `orders-api:latest` ?
3. Why was pip removed from the final image?

▼ Answers

1. **Two `FROM` lines:** this is a multi-stage build. Dependencies are installed in a build stage, and only the result is copied into a clean runtime stage, which gives a smaller, safer image.

2. **One shared tag:** Kubernetes and Argo Rollouts couldn't tell versions apart, so you couldn't compare canary and stable, roll back reliably, or know what's running.
3. **Removing pip:** it isn't needed at runtime, and its bundled libraries had known vulnerabilities that Trivy flagged. Less software means a smaller attack surface.

2. Kubernetes

What is it?

Kubernetes (often written **K8s**) runs containers on a group of machines (a **cluster**) and keeps them in the state you asked for.

You don't tell Kubernetes *how* to do things ("start a container on machine 2"). You declare *what* you want ("5 copies of orders-api must always be running"), and Kubernetes continuously works to make reality match. This is called **declarative** configuration, and the matching loop is called **reconciliation**.

The objects you need for this project

You describe what you want in YAML files. Each file describes an **object**:

Object	What it is	In this project
Pod	The smallest unit: one or more containers that run together	Each copy of orders-api is a Pod
ReplicaSet	Keeps N identical Pods running	Argo Rollouts creates one per version (stable and canary)
Deployment	Manages ReplicaSets for rolling updates	Used for the load generator, Prometheus and Grafana
Rollout (custom)	Argo Rollouts' smarter replacement for Deployment	orders-api, see chapter 7
Service	A stable name and IP that load-balances across matching Pods	orders-api (port 80) spreads traffic across all versions
Namespace	A folder that groups objects	delivery (app), monitoring (Prometheus, Grafana), argo-rollouts
ConfigMap	Configuration files stored in the cluster	Prometheus config, SLO rules, Grafana dashboard
Secret	Like a ConfigMap, but for sensitive values	Grafana admin password (generated randomly)
ServiceAccount + RBAC	An identity for a Pod, plus what it is allowed to do	Prometheus may only <i>read</i> the pod list
NetworkPolicy	A firewall between Pods	Only the same namespace and Prometheus may call orders-api

Labels hold everything together. A label is a key/value tag such as `app: orders-api`. A Service sends traffic to every Pod whose labels match its **selector**:

```
# charts/orders-api/templates/services.yaml (rendered)
kind: Service
metadata: { name: orders-api }
spec:
  selector: { app: orders-api } # every Pod labelled app=orders-api, old AND new version
  ports: [{ port: 80, targetPort: http }]
```

Because this Service selects both versions, **traffic splits by the number of Pods**: if 1 of 5 Pods runs the new version, about 20% of requests reach it. Argo Rollouts uses exactly this.

Health checks: probes

Kubernetes needs to know if a container is healthy. Look at `rollout.yaml`:

```
readinessProbe: { httpGet: { path: /readyz, port: http }, periodSeconds: 5 } # "may I receive traffic?"
livenessProbe: { httpGet: { path: /healthz, port: http }, periodSeconds: 10 } # "am I alive, or should I be restarted?"
```

- **Readiness probe failing:** the Pod is removed from the Service and gets no traffic.
- **Liveness probe failing:** the container is restarted.
- **On shutdown,** our app fails `/readyz` first, waits 5 s, then exits (see `shutdown()` in `server.py`). In-flight requests finish instead of failing. This is called **graceful shutdown**.

Resources

```
resources:
  requests: { cpu: 50m, memory: 64Mi } # guaranteed; used by the scheduler to place the Pod
  limits:   { cpu: 250m, memory: 128Mi } # maximum; above the memory limit the container is killed
```

`50m` means 50 millicores, that is 5% of one CPU core.

Security settings used here

```
securityContext: # pod level
  runAsNonRoot: true
  runAsUser: 10001
  seccompProfile: { type: RuntimeDefault } # blocks dangerous Linux system calls
containers: ... securityContext: # container level
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true # the app cannot write to its own filesystem
  capabilities: { drop: ["ALL"] } # no special Linux powers
automountServiceAccountToken: false # the app never talks to the Kubernetes API, so it gets no token
```

Where it is in this repository

File	Kubernetes objects
<code>charts/orders-api/templates/</code>	Rollout, Service, ServiceAccount, NetworkPolicy, load-generator Deployment (Helm templates, chapter 4)
<code>platform/monitoring/prometheus.yaml</code>	Namespace, ServiceAccount, ClusterRole + Binding, ConfigMap, Deployment, Service
<code>platform/monitoring/grafana.yaml</code>	ConfigMap, Deployment, Service

Try it (lab running)

```
kubectl get nodes           # the 3 machines (containers, in kind)
kubectl get ns              # namespaces
kubectl -n delivery get pods -o wide # 5 orders-api Pods + load generator, and which node
each runs on
kubectl -n delivery get pods --show-labels # see app=orders-api and rollouts-pod-template-hash
kubectl -n delivery describe svc orders-api # "Endpoints": the Pod IPs behind the Service
kubectl -n delivery delete pod <one-orders-api-pod> # watch Kubernetes replace it:
kubectl -n delivery get pods -w
kubectl -n delivery logs deploy/orders-loadgen      # status-code counts every 30 s
kubectl explain rollout.spec.strategy.canary         # built-in documentation, works for every field
```

Check yourself

1. You delete a Pod. Why does a new one appear?
2. How does the `orders-api` Service decide which Pods receive traffic?
3. What is the difference between a readiness and a liveness probe?

▼ Answers

1. **Reconciliation:** the ReplicaSet wants N Pods, sees N–1, and creates one.
2. **Label selector:** every ready Pod labelled `app: orders-api` is an endpoint, so traffic spreads across them.
3. **Readiness vs liveness:** failing readiness only removes the Pod from traffic. Failing liveness restarts the container.

3. kind (Kubernetes in Docker)

What is it?

kind creates a **real** Kubernetes cluster where every "machine" (node) is a Docker container on your computer. It was built by the Kubernetes project itself to test Kubernetes, so it is the genuine Kubernetes software, not a simulator.

Why this project uses it

Need	How kind meets it
Learn and test for free	No cloud account, no bill
Reproducible	The cluster is defined in a file (kind/cluster.yaml) and rebuilt in about 1 minute
Same setup in CI	GitHub Actions creates the identical cluster with helm/kind-action , so "works on my laptop" and "works in CI" mean the same thing
Several nodes	Canary and stable Pods spread across 2 worker nodes, closer to real life than a single node

Alternatives:

Tool	What it is	Why not used here
minikube, k3d	Other local clusters	Fine choices. kind was chosen because it's the standard for CI
EKS, GKE, AKS	Managed clusters in the cloud	They cost money. Nothing in this project depends on a specific cloud, so the same chart would run on EKS unchanged

How it is used in this project

```
# kind/cluster.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: slo-delivery
nodes:
  - role: control-plane    # runs the Kubernetes "brain" (API server, scheduler, controllers)
  - role: worker           # runs our Pods
  - role: worker
```

- [scripts/up.sh](#) runs `kind create cluster --config kind/cluster.yaml` and sets your `kubectl` to talk to it (context `kind-slo-delivery`).
- [scripts/build-image.sh](#) runs `kind load docker-image orders-api:<version>` , which copies the image into every node. That's why no registry is needed.
- `imagePullPolicy: IfNotPresent` in the chart tells Kubernetes to use the loaded image rather than try to download it.

Networking: kind's default network plugin (kindnet) enforces NetworkPolicy. `scripts/check-network-policy.sh` proves it.

Try it

```
kind get clusters           # slo-delivery
docker ps --filter name=slo-delivery # the 3 "nodes" are just containers
kubectl config current-context # kind-slo-delivery
make down && make up        # destroy and rebuild everything (about 3 min)
```

Check yourself

1. Is a kind cluster "fake Kubernetes"?
2. Why does the lab not need Docker Hub or another registry?

▼ Answers

1. **No.** It runs the real Kubernetes components; only the nodes are containers instead of VMs.
2. **No registry needed:** `kind load docker-image` copies locally built images directly into the nodes, and `imagePullPolicy: IfNotPresent` makes Kubernetes use them.

4. Helm

What is it?

Helm is the package manager for Kubernetes. A **chart** is a folder of Kubernetes YAML **templates** plus a **values.yaml** file of settings. Helm fills the templates with the values and sends the result to the cluster. Each install or upgrade is recorded as a **release revision**, so you can see the history and roll back.

```
values.yaml + templates/*.yaml  —helm template→  plain Kubernetes YAML  →  cluster
(settings)      (with {{ }} gaps)
```

Why this project uses it

1. **One source, many configurations.** The same chart runs with real timings (`values.yaml` : 60 s pauses) and fast test timings (`ci/fast-values.yaml` : 30 s pauses). No copy-pasted YAML.
2. **Computed values.** The canary error limit is **computed** from the SLO, not typed in by hand (see below).
3. **A release is one command.** `helm upgrade --set image.tag=1.2.0` changes the version; Argo Rollouts takes it from there.

How it works in this project

The chart: `charts/orders-api/`

<code>Chart.yaml</code>	name and version of the chart
<code>values.yaml</code>	all settings with comments (SLO targets, analysis timing, canary steps, faults)
<code>templates/</code>	
<code>_helpers.tpl</code>	reusable snippets: labels, security contexts, the computed threshold
<code>rollout.yaml</code>	the Argo Rollouts canary (chapter 7)
<code>analysistemplate.yaml</code>	the SLO checks (chapter 7)
<code>services.yaml</code>	the client-facing Service
<code>networkpolicy.yaml</code>	the firewall rule
<code>serviceaccount.yaml</code>	
<code>loadgen.yaml</code>	steady test traffic

Template syntax in 30 seconds:

```
replicas: {{ .Values.replicas }}                # insert a value
image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
{{- range .Values.canary.weights }}              # loop: one step per weight
- setWeight: {{ . }}
{{- end }}
{{- if .Values.networkPolicy.enabled }} ... {{- end }} # optional object
labels: {{- include "orders-api.labels" . | nindent 4 }} # reuse a snippet from _helpers.tpl
```

The **computed threshold** (in `_helpers.tpl`) turns business numbers into the technical limit:

```
{{- printf "%.4f" (mulf (subf 1.0 .Values.slo.availabilityTarget) .Values.analysis.maxBurnRate) -}}
(1 - 0.995) × 14.4 = 0.0720
```

If someone changes the SLO to 99.9%, the canary limit becomes stricter automatically (0.0144).

An escaping trick worth knowing: the analysis query needs the literal text `{{args.canary-hash}}`, which Argo Rollouts fills in later. Because Helm would try to interpret `{{ }}` itself, the template writes `{{ "{{" }}args.canary-hash{{ " }}" }}`.

A real problem found in this project (Helm 4): Helm 4 uses **server-side apply**, so it "owns" every field it writes. When Argo Rollouts also changed the selector of extra stable/canary Services, the next `helm upgrade` failed with a field-ownership conflict. The fix was to remove those Services. Read [troubleshooting](#).

Try it

```
helm lint charts/orders-api                # checks the chart
helm template orders-api charts/orders-api | less      # see the final YAML without installing
helm template orders-api charts/orders-api -f ci/fast-values.yaml | grep -A2 "pause"
helm template orders-api charts/orders-api --set slo.availabilityTarget=0.999 | grep successCondition
helm -n delivery list                      # installed releases
helm -n delivery history orders-api        # every release revision
helm -n delivery get values orders-api     # the values currently in use
```

Check yourself

1. What does `helm template` do, and why is it useful in CI?
2. Where does the number 0.072 come from? What happens to it if the SLO becomes 99.9%?
3. What does `--reuse-values` do in `scripts/release.sh`?

▼ Answers

1. `helm template` renders the final YAML locally without touching a cluster. CI pipes that output into kubeconform to validate it against Kubernetes schemas.
2. **0.072** is $(1 - 0.995) \times 14.4$. At 99.9% it becomes $(1 - 0.999) \times 14.4 = 0.0144$, a stricter gate.
3. `--reuse-values` keeps the values from the previous release and only changes what you `--set`, such as the image tag.

5. SRE basics: SLI, SLO, error budget, burn rate

These four ideas come from Google's **Site Reliability Engineering (SRE)** practice. They turn "the service feels slow" into numbers everybody agrees on. This project is built on them.

SLI: Service Level Indicator, "what we measure"

An SLI is a measurement of user experience, usually as a ratio of good events to all events.

SLI in this project	Formula
Availability	requests that did not return a 5xx error ÷ all requests
Latency	requests that finished in under 300 ms ÷ all requests

Why ratios? They don't depend on traffic volume: 1 error in 1,000 is the same quality at night as at peak.

SLO: Service Level Objective, "how good is good enough"

An SLO is a target for an SLI over a time window.

SLO	Meaning
Availability 99.5% (30 days)	At most 0.5% of requests may fail
Latency 99% < 300 ms	At most 1% of requests may be slower than 300 ms

100% is never the target. It's impossibly expensive, and users can't tell 99.99% from 100% anyway.

An **SLA** (Service Level Agreement) is different: it's a contract with customers, usually with financial penalties. SLOs are internal and stricter than any SLA.

Error budget: "how much failure we can afford"

Error budget = 1 – SLO. With 99.5%, the budget is **0.5%** of requests. The budget is something you are *allowed to spend* on releases, experiments and incidents. - **Budget left:** ship faster. - **Budget gone:** slow down and fix reliability.

In this repo the recording rule `slo:error_budget_remaining:ratio` computes `1 - (error_ratio ÷ 0.005)`. - **1** means untouched. - **0** means fully spent. - **Negative** means overspent.

Burn rate: "how fast we are spending the budget"

Burn rate = current error ratio ÷ error budget.

Burn rate	Meaning
1x	Spending exactly as planned; the budget lasts the whole 30 days
14.4x	The 30-day budget would be gone in about 2 days. 2% of it disappears every hour
0.5x	Well within budget

Example: the error ratio is 7.2% and the budget is 0.5%, so the burn rate is $7.2 \div 0.5 = 14.4x$.

Multi-window, multi-burn-rate alerting

The naive alert, "page if errors > 0.5% for 5 minutes", is bad: - **Too noisy**: a 5-minute blip pages someone. - **Too slow** for disasters, and it says nothing about *how much* budget is at risk.

The SRE Workbook approach, used in [slo/rules/orders-api-slo.rules.yaml](#) :

Alert	Fires when the burn rate is above...	...over BOTH windows	Severity
Fast burn	14.4x	1 hour and 5 minutes	page (wake someone up)
Slow burn	6x	6 hours and 30 minutes	page
Ticket burn	3x	1 day and 2 hours	ticket (fix during work hours)

Why two windows: - The **long** window proves the problem is significant (not a blip). - The **short** window proves it is **still happening**, so the alert stops quickly after a fix instead of firing for another hour.

How this project uses these ideas for releases

The key design decision: a canary may burn the error budget **at most 14.4x** faster than sustainable. That's the same number as the fast-burn page:

```
canary limit = error budget × max burn rate = 0.005 × 14.4 = 0.072 (7.2% errors)
```

In plain words: **a version that would page the on-call engineer if it served all users must never be promoted**. Release safety and alerting use one shared definition of "unhealthy".

One more case: a regression that is too small for the canary gate (for example 4% errors) still gets caught. Over hours it triggers the **slow-burn** page, which the `promtool` tests prove.

Try it

Open Grafana (`kubectl -n monitoring port-forward svc/grafana 3000:3000`) and find: - **Availability SLI (5m) - Error budget remaining - Burn rate (1h)**

Then run `scripts/release.sh 1.9.0 --inject-errors` and watch the burn rate rise and the error budget fall while the canary runs. Then watch the canary disappear.

Check yourself

1. The SLO is 99.9%. What is the error budget, and what error ratio is a 6x burn?
2. Why does each alert need two windows?
3. Why is the canary limit 14.4x the budget, and not, say, 1x?

▼ Answers

1. **Budget** = 0.1%. A 6x burn is an error ratio of 0.6%.
2. **Two windows**: the long window makes sure the problem is significant; the short window makes sure it is still ongoing, so alerts resolve quickly.
3. **14.4x, not 1x**: a canary only runs for minutes, so judging it at 1x would be extremely strict and noisy. 14.4x means "would trigger a page", which is exactly the line a release must not cross.

Smaller regressions are caught later by the slow-burn alert.

6. Prometheus and PromQL

What is it?

Prometheus is a monitoring system that **collects numbers over time** (time series) and lets you **query** them. Every few seconds it visits each application's `/metrics` page (called **scraping**) and stores what it reads.

```
orders-api pod — GET /metrics every 5 s → Prometheus → stores, queries, alerts
http_requests_total{route="/api/orders",code="200"} 1234
http_requests_total{route="/api/orders",code="500"} 7
```

Why this project uses it

Prometheus is the single source of truth for the SLIs. It does four jobs here: 1. **Collects** request counts and latencies from every Pod, labelled with the Pod's **version**. 2. **Records** the SLIs over several windows (recording rules). 3. **Alerts** on error-budget burn rate (alerting rules). 4. **Answers the canary analysis.** Argo Rollouts sends it a query every 30 s.

Alternatives: Datadog, New Relic (hosted, paid), OpenTelemetry + a backend. Prometheus is the open-source standard for Kubernetes and the native data source for Argo Rollouts' analysis.

Metric types used by the app

From `app/orders_api/server.py`:

Metric	Type	Meaning
<code>http_requests_total{route,method,code}</code>	Counter: only goes up	Total requests, split by route and status code
<code>http_request_duration_seconds{route}</code>	Histogram: counts per time "bucket"	How many requests took ≤ 25 ms, ≤ 50 ms, ... ≤ 300 ms, ... ≤ 2.5 s
<code>app_info{version}</code>	Gauge: can go up and down	Constant 1, with the version as a label

Labels (the `{...}` part) let you filter and group. Each unique combination of labels is one **time series**.

Service discovery and relabelling: the clever part

Prometheus has to find the Pods on its own, because they come and go during every release.

`platform/monitoring/prometheus.yaml` uses `kubernetes_sd_configs` (role: pod) and then **relabelling** rules:

```
- source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
  action: keep
  regex: "true" # only Pods annotated prometheus.io/scrape: "true"
- source_labels: [__meta_kubernetes_pod_label_rollouts_pod_template_hash]
  target_label: rollouts_pod_template_hash # ← THE key line
```

Argo Rollouts gives every version's Pods a label `rollouts-pod-template-hash` (for example `7f6866c7fc`). By copying it onto every sample, Prometheus can answer "error ratio of the **new** version only". Without this line,

canary analysis would be impossible.

RBAC: Prometheus' ClusterRole allows only `get/list/watch` on Pods. That's the least privilege needed for discovery.

PromQL: the query language, step by step

```
http_requests_total                # every series of this counter
http_requests_total{route="/api/orders",code=~"5.."} # filter: =~ is a regex match, so any 5xx
rate(http_requests_total[1m])      # per-second increase over the last minute
sum(rate(http_requests_total[1m])) # add all series: total requests/s
sum by (version) (rate(http_requests_total[1m]))    # requests/s per version (the traffic split!)
```

The availability SLI (error ratio):

```
(sum(rate(http_requests_total{route="/api/orders",code=~"5.."}[5m])) or vector(0))
/
sum(rate(http_requests_total{route="/api/orders"}[5m]))
```

`or vector(0)` handles the case where no 5xx has ever happened, so the numerator series doesn't exist yet. Without it, the result would be empty instead of 0.

The p95 latency from the histogram:

```
histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[1m])))
```

Accuracy note: the result is interpolated inside a bucket, so its precision depends on the bucket boundaries. With +600 ms injected, the lab measured 0.975 s, because the true value sits in the 0.5–1 s bucket.

Recording rules and alerting rules

Recording rules pre-compute expensive queries and give them a name. The convention is

`level:metric:operation`:

```
- record: slo:sli_error:ratio_rate1h
  expr: <the error-ratio query with [1h]>
```

Alerting rules fire when an expression is true for a duration (`for:`):

```
- alert: OrdersApiErrorBudgetFastBurn
  expr: slo:sli_error:ratio_rate1h > (14.4 * 0.005) and slo:sli_error:ratio_rate5m > (14.4 * 0.005)
  for: 2m
```

The rules file is the single source of truth. `scripts/up.sh` turns `slo/rules/` into a ConfigMap, and the same file is unit-tested by `promtool` (chapter 10).

Try it

```
kubect1 -n monitoring port-forward svc/prometheus 9090:9090 # open http://localhost:9090
```

In the Prometheus UI: - **Status** → **Targets:** every orders-api Pod, with its `rollouts_pod_template_hash` - **Graph:** run the queries above one by one, then `slo:sli_error:ratio_rate5m` - **Alerts:** the 4 SLO alerts and their state

Start a bad release (`scripts/release.sh 1.9.0 --inject-errors`) and query:

```
sum by (version) (rate(http_requests_total{code=~"5.."}[1m])) / sum by (version)
(rate(http_requests_total[1m]))
```

Check yourself

1. Why is `rate()` applied to a counter instead of using the raw value?
2. What would break if the relabel rule for `rollouts_pod_template_hash` were deleted?
3. What does `for: 2m` add to an alert?

▼ Answers

1. **`rate()` on counters:** a counter only grows, and resets when a Pod restarts. `rate()` gives the per-second speed and handles resets.
2. **Without the relabel rule:** the analysis queries could no longer separate canary Pods from stable Pods. They would return no data, so every rollout would fail safe.
3. **`for: 2m`:** the condition must stay true for 2 minutes before the alert fires, which filters out short spikes.

7. Progressive delivery and Argo Rollouts

What is progressive delivery?

Progressive delivery releases a new version to a **small share of users first**, measures it, and only continues if it is healthy.

Strategy	How it works	Risk
Recreate	Stop the old version, start the new	Downtime; everybody is hit by bugs
Rolling update (Kubernetes default)	Replace Pods a few at a time	No downtime, but it doesn't <i>check health</i> , so a bad version still reaches 100% in minutes
Blue/green	Run the full new version next to the old one, switch all traffic at once	Fast rollback, but an all-or-nothing switch and double the resources
Canary ← this project	Send a small, growing share of traffic to the new version, check metrics at each step	Bad versions reach few users, and only briefly

The name comes from the canaries miners took into coal mines to detect gas early.

What is Argo Rollouts?

Argo Rollouts is a Kubernetes **controller**: a program running in the cluster (namespace `argo-rollouts`) that adds new object types (**CRDs**, Custom Resource Definitions):

Object	Role
Rollout	Replaces a Deployment. Same Pod template, plus a <code>strategy.canary</code> section describing the steps
AnalysisTemplate	A reusable definition of <i>what to measure</i> and <i>what counts as success</i>
AnalysisRun	One execution of a template during a specific release, with every measurement stored

Why this tool: Kubernetes Deployments can't pause, measure, or roll back on metrics. Argo Rollouts adds exactly that, with native Prometheus support.

Alternatives: Flagger (similar, often with a service mesh); feature-flag platforms (release *features* rather than *deployments*); hand-written pipeline scripts (fragile).

How it works in this project

The steps (`rollout.yaml`)

```

strategy:
  canary:
    analysis:
      # BACKGROUND analysis: runs from step 1 until full promotion
      startingStep: 1
      templates: [{ templateName: orders-api-slo }]
      args:
        - name: canary-hash
          valueFrom: { podTemplateHashValue: Latest } # the new version's pod hash
    steps:
      - setWeight: 20 # 1 of 5 pods runs the new version
      - pause: { duration: 60s }
      - setWeight: 40
      - pause: { duration: 60s }
      - setWeight: 60
      - pause: { duration: 60s }
      - setWeight: 80
      - pause: { duration: 60s } # after the last step: promoted to 100%

```

No traffic router: - `setWeight: 20` with 5 replicas means **1 canary Pod**. - The single Service spreads traffic across all Pods, so the canary gets about 20%. - **Trade-off:** steps can only be multiples of 1/replicas. A traffic router (Gateway API, Istio, NGINX) would allow exact 1% or 5% steps.

The checks (`analysis-template.yaml`)

```

metrics:
  - name: canary-error-ratio
    interval: 30s # measure every 30 s
    failureLimit: 1 # 2 failures → the analysis fails → the rollout aborts
    consecutiveErrorLimit: 4 # 5 query ERRORS in a row (Prometheus down) → abort
    successCondition: "len(result) > 0 && !isNaN(result[0]) && result[0] <= 0.0720"
    provider:
      prometheus:
        address: http://prometheus.monitoring.svc.cluster.local:9090
        query: <error ratio, filtered with rollouts_pod_template_hash="{{args.canary-hash}}">
  - name: canary-p95-latency # same idea, success if p95 ≤ 0.3 s

```

Reading `successCondition`: - `len(result) > 0`: Prometheus returned data. **No data means fail**, because a canary nobody used is not verified. - `!isNaN(result[0])`: the value is a real number (dividing by zero traffic gives NaN). - `result[0] <= 0.0720`: within the SLO-based limit.

The three outcomes

Analysis result	What Argo Rollouts does
Successful at every check	Continues through all steps, then promotes: the new version becomes stable
Failed (SLO breached twice)	Aborts: the canary scales to 0, stable returns to 5 Pods, users get the old version
Error (cannot measure 5 times in a row)	Aborts too: never promote what you can't verify (fail safe)

After an abort: - the Rollout shows `abort: true` and phase `Degraded` - you fix the code and release a new version, or point the spec back to the stable version

Measured in this project (kind lab)

Release	Decision
Healthy 1.1.0	9 of 9 checks passed, promoted after 2 min 46 s
25% errors	Aborted after 47 s (measured 21.7% and 19.8% against the 7.2% limit)
+600 ms	Aborted after 46 s (p95 0.975 s against the 0.3 s limit)
Prometheus outage	Analysis Error , aborted after 60 s

Try it

```
kubectl -n delivery get rollout orders-api -w          # watch phase and step change
scripts/release.sh 2.0.0                             # healthy release
kubectl -n delivery get rs                             # two ReplicaSets during the canary: old and
new
kubectl -n delivery get analysisrun                   # one run per release
kubectl -n delivery get analysisrun <name> -o yaml | sed -n '/^status:/$p' # every measurement
scripts/release.sh 2.1.0 --inject-errors              # watch the abort happen
```

Optional kubectl plugin for a nice tree view: install `kubectl-argo-rollouts`, then `kubectl argo rollouts get rollout orders-api -n delivery --watch`.

Check yourself

1. Why does the analysis query filter by `rollouts_pod_template_hash`?
2. What happens if no traffic reaches the canary?
3. Why is a Prometheus outage treated as "abort", not "continue"?

▼ Answers

1. **The hash filter:** to judge only the new version's Pods. A service-wide average at 20% canary weight would dilute a broken canary with healthy stable traffic.
2. **No traffic:** the query returns no data, `len(result) > 0` fails, and the release is not promoted.
3. **Fail safe:** promoting a release without evidence that it is healthy is exactly the risk this system exists to remove.

8. Grafana

What is it?

Grafana turns numbers into dashboards: charts, single-value "stat" panels, tables. It doesn't store data itself. It asks a **data source** (here Prometheus) using queries and draws the answers.

Why this project uses it

Prometheus can answer any question, but humans need to see a release happening.

The dashboard "orders-api: SLOs & progressive delivery" shows at a glance: - **SLO state:** availability, error budget remaining, burn rate, latency, firing alerts. - **Canary vs stable:** error ratio and p95 **per version**. - **Traffic split:** the share of traffic per version, moving step by step during a rollout or dropping to 0 on abort. **Alternatives:** the Prometheus UI (basic graphs), Kibana (logs-first), Datadog dashboards (hosted, paid).

Dashboards as code

Nothing is clicked together by hand. Grafana is **provisioned from files**, so the dashboard is version-controlled and rebuilt identically every time:

File	What it provides
<code>platform/monitoring/grafana.yaml</code> → <code>datasources.yaml</code>	The Prometheus data source (uid <code>prometheus</code>)
same file → <code>dashboards.yaml</code>	Tells Grafana to load every JSON dashboard from a folder
<code>grafana/dashboards/orders-api-slo.json</code>	The dashboard: 9 panels, each with its PromQL query

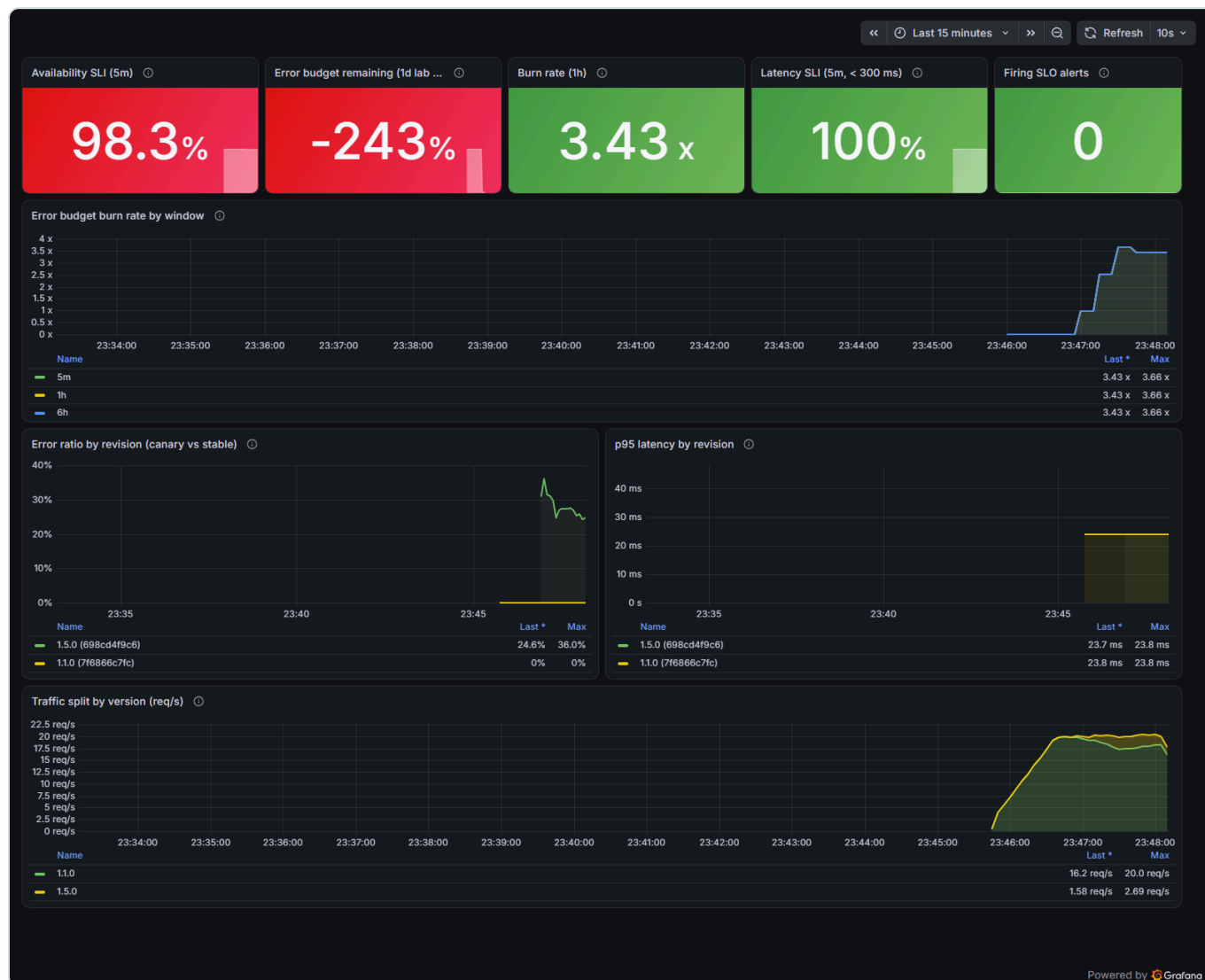
`scripts/up.sh` turns the JSON into a ConfigMap and mounts it into the Grafana Pod.

The panels and what to look for

Panel	Query idea	What it tells you
Availability SLI (5m)	<code>1 - slo:sli_error:ratio_rate5m</code>	Green $\geq 99.5\%$
Error budget remaining	<code>slo:error_budget_remaining:ratio</code>	Below 0 means the budget is overspent
Burn rate (1h)	<code>slo:sli_error:ratio_rate1h / 0.005</code>	Red at $\geq 14.4x$ (the fast-burn page)
Burn rate by window	5m, 1h and 6h lines, threshold lines at 6x and 14.4x	The multi-window picture
Error ratio by revision	error ratio grouped by <code>version</code>	Canary vs stable side by side
p95 latency by revision	<code>histogram_quantile</code> grouped by <code>version</code>	Latency regressions per version

Panel	Query idea	What it tells you
Traffic split by version	requests/s grouped by <code>version</code>	Watch the canary share grow, or vanish on abort

A lesson from this project: a version with **zero** errors had no 5xx series, so it vanished from the error-ratio panel, which is exactly the version you want to see at 0%. The fix adds `or <total traffic> * 0` so every version always appears.



Security note

- **Viewing:** the lab enables **read-only anonymous** access, so you can view the dashboard without logging in.
- **Admin password:** random, created at deploy time, stored only as a Kubernetes Secret.
- **To read the password:** `bash kubectl -n monitoring get secret grafana-admin -o jsonpath='{.data.password}' | base64 -d`

Try it

```
kubectl -n monitoring port-forward svc/grafana 3000:3000      # http://localhost:3000
scripts/release.sh 3.0.0 --inject-errors                     # watch the canary spike and disappear
```

In Grafana: - open any panel → **Edit** to see its PromQL - change the time range to "Last 15 minutes" - hover the charts to compare the canary and stable lines

Check yourself

1. Where does Grafana get its data?
2. Why is the dashboard stored as JSON in Git instead of built in the UI?

▼ Answers

1. **From Prometheus**, through the provisioned data source. Grafana only queries and draws.
2. **JSON in Git**: it is reproducible, reviewable and versioned. A rebuilt cluster gets the identical dashboard, and changes go through pull requests.

9. GitHub Actions (CI/CD)

What is it?

CI (Continuous Integration) means automatically building and testing every code change. **CD (Continuous Delivery/Deployment)** means automatically releasing changes that passed.

GitHub Actions is GitHub's built-in automation. You write a **workflow** (YAML in `.github/workflows/`) that runs on GitHub's machines (**runners**) when something happens, for example a push.

Term	Meaning
Workflow	The whole automation file (<code>ci.yaml</code>)
Trigger (<code>on:</code>)	When it runs: here on push to <code>main</code> , on pull requests, and manually
Job	A group of steps on one fresh machine. Jobs run in parallel unless they <code>need</code> each other
Step	One command (<code>run:</code>) or a reusable action (<code>uses:</code>)
Action	A packaged step from the marketplace, for example <code>actions/checkout</code> , <code>helm/kind-action</code>

Why this project uses it

Every change must prove that the safety net still works. A pipeline that only runs unit tests could miss a broken Prometheus rule or a broken rollback. This pipeline runs the **real release scenarios on a real cluster** for every push.

Alternatives: Jenkins (self-hosted, very flexible), GitLab CI, CircleCI. GitHub Actions is used because the code lives on GitHub and kind runs well on its runners.

The pipeline in this project

```
push → job "static" ───┐
    │                  │   pytest, ruff, shellcheck, promtool check + test, helm lint, kubeconform
    │                  └─┘
    │                  → job "image" ───┐
    │                  │                  │ docker build, Trivy scan (fails on fixable HIGH/CRITICAL)
    │                  └─┘                  │
    │                  ──────────────────┘ job "e2e" (needs both; runs only if they pass)
                                          create kind cluster → scripts/up.sh
                                          → scripts/e2e.sh (good release promoted, 2 bad ones rolled back)
                                          → scripts/chaos-prometheus-outage.sh
                                          → scripts/check-network-policy.sh
                                          on failure: collect diagnostics → upload as artifact
```

Good practices to notice in `ci.yaml` :

Practice	Line	Why
Least-privilege token	<code>permissions: contents: read</code>	The workflow can't push code or change settings
Cancel outdated runs	<code>concurrency: ... cancel-in-progress: true</code>	A new push stops the old, now-pointless run

Practice	Line	Why
Pinned versions	<code>prom/prometheus:v3.15.0</code> , <code>kubeconform v0.8.0</code> , <code>trivy:0.75.0</code>	Same result today and next month
Fail fast, expensive last	<code>needs: [static, image]</code>	The 15-minute cluster test runs only if the cheap checks pass
Debuggable failures	<code>if: failure()</code> → upload diagnostics	You get the Rollout, AnalysisRun and controller logs without re-running
Same scripts as local	<code>scripts/up.sh</code> , <code>scripts/e2e.sh</code>	"Passes on my laptop" and "passes in CI" mean the same thing

Two real CI lessons from this project: 1. **ShellCheck versions:** the runner's ShellCheck version flagged a pattern (`A && B || C` , SC2015) that the local version didn't, so the code was fixed. 2. **The executable bit:** the scripts were created on Windows without it, so Linux refused to run them (`Permission denied`). Fixed with `git update-index --chmod=+x scripts/*.sh` .

Try it

On GitHub: open the repository's **Actions** tab, open a run, then open each job and step. You can see every command's output.

Locally: the Makefile runs the same checks:

```
make test lint slo-test      # the "static" job
make image                  # the build part of the "image" job
make e2e chaos netpol       # the "e2e" job (lab must be up)
```

Check yourself

1. Why does the `e2e` job declare `needs: [static, image]` ?
2. Why are tool versions pinned?
3. What is the benefit of CI calling the same scripts you run locally?

▼ Answers

1. **needs** : it's slow and expensive, so it only runs when the fast checks have passed. That gives quick feedback for simple mistakes.
2. **Pinned versions:** they make results reproducible. An unpinned tool can change behaviour overnight and break or silently weaken the pipeline.
3. **Same scripts locally and in CI:** one implementation, no drift. If it passes locally, CI runs exactly the same steps.

10. Quality and security tools

These small tools each catch one class of mistake **before** it reaches a cluster. All of them run in CI.

Tool	Checks	Catches, for example	Run it
pytest	Python unit tests	Fault injection producing the wrong error rate	<code>make test</code>
ruff	Python lint and format	Unused imports, insecure patterns, style	<code>cd app && ruff check .</code>
ShellCheck	Bash scripts	Unquoted variables, <code>A && B \ C</code> traps	<code>shellcheck scripts/*.sh</code>
promtool	Prometheus rules	Syntax errors, and alerts that don't fire when they should	<code>make slo-test</code>
helm lint	Helm chart	Template errors, missing fields	<code>helm lint charts/orders-api</code>
kubeconform	Kubernetes YAML against official schemas	Typos such as <code>replica:</code> vs <code>replicas:</code> , wrong types	<code>make lint</code>
Trivy	Container image vulnerabilities	Known CVEs in OS packages and libraries	see below

pytest: unit tests for the app

`app/tests/test_server.py` tests the logic without starting a server: - metrics are counted per route and code - probes are not counted as traffic - `FAULT_ERROR_RATE=0.25` produces between 22% and 28% errors over 2,000 requests - invalid configuration is rejected

A design choice that makes testing easy: `App.handle()` is plain Python that returns `(status, content_type, body)`, separate from the HTTP server.

promtool: unit tests for alerts (rarely done, very valuable)

Most teams never test their alerts. They find out during an incident that an alert never fires.

`slo/tests/orders-api-slo.test.yaml` feeds **synthetic time series** into the real rules and asserts the outcome:

```
- name: fast burn pages within minutes
  input_series:
    - series: 'http_requests_total{app="orders-api",route="/api/orders",method="GET",code="200"}'
      values: '0+90x120'          # starts at 0, +90 every minute, for 120 minutes
    - series: '...code="500"}'
      values: '0+10x120'          # +10 per minute, so 10% errors
  alert_rule_test:
    - eval_time: 10m
      alertname: OrdersApiErrorBudgetFastBurn
      exp_alerts: [ { exp_labels: { severity: page, ... } } ] # MUST be firing
```

The four tests prove: - **10% errors:** fast burn pages. - **4% errors:** slow burn pages, but fast burn doesn't. - **0.1% errors:** nothing fires, and 80% of the budget is left. - **Slow requests:** the latency alert fires.

kubeconform: schema validation

```
helm template orders-api charts/orders-api | kubeconform -strict -summary \
  -schema-location default \
  -schema-location 'https://raw.githubusercontent.com/datreeio/CRDs-
  catalog/main/{{.Group}}/{{.ResourceKind}}_{{.ResourceAPIVersion}}.json'
```

- **-strict** rejects unknown fields, which catches typos.
- **The second schema location** provides schemas for **custom** resources such as Argo Rollouts' `Rollout` and `AnalysisTemplate`, which plain Kubernetes doesn't know.

Trivy: image vulnerability scanning

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:0.75.0 \
  image --exit-code 1 --ignore-unfixed --severity HIGH,CRITICAL orders-api:1.1.0
```

Flag	Why
<code>--severity HIGH,CRITICAL</code>	Focus on serious issues
<code>--ignore-unfixed</code>	Only fail on vulnerabilities that have a fix, so the pipeline stays actionable
<code>--exit-code 1</code>	Makes CI fail when something is found

What happened in this project: the first scan found **5 HIGH** vulnerabilities. None were in the app's own dependency: - `urllib3` and `msgpack`, inside **pip** - `setuptools` - an OS package (`libpcre2`)

The fix was architectural, not a suppression: - remove `pip` and `setuptools` from the runtime image - apply OS security updates

Result: **0 findings**. That's the DevSecOps habit: fix the cause, don't silence the scanner.

Check yourself

1. What does `promtool test` (not `check`) give you that a syntax check doesn't?
2. Why `--ignore-unfixed` in CI?
3. Why does `kubeconform` need an extra schema location for this project?

▼ Answers

1. **Behaviour:** proof that alerts fire under the right conditions, at the right time, and stay silent otherwise.
2. `--ignore-unfixed` : failing on vulnerabilities nobody can fix yet would block every build without giving anyone an action to take.
3. **The extra schema location:** `Rollout` and `AnalysisTemplate` are Custom Resources. Their schemas come from a CRD catalog, not from Kubernetes itself.

11. How everything fits together

This chapter follows **one request** and then **one release** through every tool, so you can see how each piece hands work to the next.

A. The life of one request

- | | |
|--|--------------------------------------|
| 1. orders-loadgen Pod sends GET http://orders-api/api/orders | (load generator, 20 req/s) |
| 2. Kubernetes DNS resolves "orders-api" to the Service IP | (Kubernetes Service) |
| 3. The Service picks one ready Pod labelled app=orders-api | (old OR new version) |
| 4. NetworkPolicy allows it: the source is in the same namespace | (Kubernetes NetworkPolicy) |
| 5. orders-api handles it and increments
http_requests_total{route="/api/orders",code="200"}
http_request_duration_seconds_bucket{le="0.025",...} | (Python + prometheus-client) |
| 6. Every 5 s, Prometheus scrapes that Pod's /metrics, adding labels
app, version, rollouts_pod_template_hash, pod
relabelling) | (Prometheus discovery + relabelling) |
| 7. Every 15 s, recording rules update slo:sli_error:ratio_rate5m, ... | (Prometheus rules) |
| 8. Grafana queries those series and draws the charts | (Grafana) |

B. The life of one release

```
t=0  You run: scripts/release.sh 1.2.0 --inject-errors
      ├── docker build -t orders-api:1.2.0                (Docker)
      ├── kind load docker-image orders-api:1.2.0         (kind)
      └─ helm upgrade --set image.tag=1.2.0               (Helm: the Rollout's Pod template changes)

t=1s  Argo Rollouts notices the new Pod template          (Argo Rollouts controller)
      ├── creates a new ReplicaSet with hash 6bb66c87ff
      ├── setWeight 20 → 1 canary Pod + 4 stable Pods
      └─ starts AnalysisRun "orders-api-6bb66c87ff-3"
         with arg canary-hash=6bb66c87ff

t=10s The canary Pod is ready; the Service starts sending it about 20% of requests
      └─ 25% of its responses are 500 (injected fault)

t=20s First measurement: the AnalysisRun asks Prometheus (Argo Rollouts → Prometheus)
      error ratio of pods with rollouts_pod_template_hash="6bb66c87ff" → 0.217
      0.217 ≤ 0.072? NO → failed (1 of 2 allowed)

t=35s Second measurement → 0.198 → failed (2) > failureLimit (1)
      └─ AnalysisRun phase = Failed

t=47s Argo Rollouts ABORTS the rollout
      ├── canary ReplicaSet scaled to 0
      ├── stable ReplicaSet back to 5 Pods
      └─ Rollout: abort=true, phase=Degraded, message names the failed metric

after Clients only get 1.1.0 again (verified by e2e.sh)
    Grafana: the canary error line spiked, then its traffic dropped to 0
    Burn-rate alerts: no page, because only 20% of traffic was exposed for under a minute
```

These timings are the real measurements from the lab run (`ci/fast-values.yaml` , 15 s interval). See [docs/test-results.md](#).

C. Why each tool is there (and what would happen without it)

Remove...	What breaks
Docker	Nothing to deploy; versions aren't immutable artifacts
Kubernetes	No self-healing, no Services, no Pods for Argo Rollouts to manage
kind	You'd need a paid cloud cluster to practise and test
Helm	Copy-pasted YAML per environment; the threshold would no longer be computed from the SLO
Prometheus	No SLIs: the analysis has nothing to measure, so every rollout aborts (fail safe)
The <code>rollouts_pod_template_hash</code> relabel	Canary and stable are indistinguishable; analysis returns no data
Argo Rollouts	Back to rolling updates: bad versions reach 100% with no automatic decision
Grafana	The system still works, but humans can't see or explain what happened
GitHub Actions	Nobody notices when a change breaks the safety net itself
promtool tests	An alert could silently never fire, and you'd find out during an incident

D. Files, mapped to concepts

Concept	File
SLI metrics in code	<code>app/orders_api/server.py</code>
SLO numbers	<code>charts/orders-api/values.yaml</code> (<code>slo:</code>), <code>slo/rules/orders-api-slo.rules.yaml</code>
Canary steps	<code>charts/orders-api/templates/rollout.yaml</code>
Release gate	<code>charts/orders-api/templates/analysistemplate.yaml</code>
Version-aware scraping	<code>platform/monitoring/prometheus.yaml</code> (<code>relabel_configs</code>)
Burn-rate alerts	<code>slo/rules/orders-api-slo.rules.yaml</code>
Alert tests	<code>slo/tests/orders-api-slo.test.yaml</code>
Dashboard	<code>grafana/dashboards/orders-api-slo.json</code>
Full automation	<code>scripts/*.sh</code> , <code>Makefile</code> , <code>.github/workflows/ci.yaml</code>

12. Hands-on labs

Do these in order with the lab running (`make up`). Each lab says what to do, what you should see, and what you learned. Open three terminals:

```
# terminal 1: watch the rollout
kubectl -n delivery get rollout orders-api -w
# terminal 2: Grafana (http://localhost:3000)
kubectl -n monitoring port-forward svc/grafana 3000:3000
# terminal 3: your commands
```

Lab 1: Explore the running system (15 min)

1. `kubectl get pods -A` : find the Pods for the app, the load generator, Prometheus, Grafana and Argo Rollouts.
2. `kubectl -n delivery get pods --show-labels` : find the `rollouts-pod-template-hash` label.
3. `kubectl -n delivery logs deploy/orders-loadgen` : see the responses by status code.
4. In Grafana, find the current availability and the traffic per version.

✅ **You learned:** how the pieces are laid out across namespaces, and how labels identify a version.

Lab 2: Release a healthy version (10 min)

```
scripts/release.sh 1.6.0
```

Watch terminal 1: the phase goes `Progressing` → `Paused` (at each step) → `Healthy` .

```
kubectl -n delivery get rs                # during the canary: two ReplicaSets (old and new)
kubectl -n delivery get analysisrun       # Running, then Successful
```

In Grafana, *Traffic split by version* shows 1.6.0 growing from 20% to 100%.

✅ **You learned:** a canary moves step by step while measurements keep passing.

Lab 3: Release a broken version (10 min)

```
scripts/release.sh 1.7.0 --inject-errors
```

Expect: - within about a minute, terminal 1 shows the rollout aborted - the canary Pod disappears

```
run=$(kubectl -n delivery get analysisrun --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1:].metadata.name}')
kubectl -n delivery get analysisrun "$run" -o jsonpath='{range .status.metricResults[*]}{.name}:
{.phase} {.measurements[*].value}{"\n"}{end}'
kubectl -n delivery get rollout orders-api -o jsonpath='{.status.message}{"\n"}'
```

To clear the abort, point the release back at the stable version:

```
helm upgrade orders-api charts/orders-api -n delivery --reuse-values --set image.tag=1.6.0 \
--set-string fault.errorRate=0 --set-string fault.latencyMs=0
```

✅ **You learned:** a measured SLO breach stops a release automatically, and users keep the stable version.

Lab 4: Latency regression (10 min)

```
scripts/release.sh 1.8.0 --inject-latency
```

Expect: this time `canary-p95-latency` fails and `canary-error-ratio` passes.

Question: why does the p95 read 0.975 s instead of 0.62 s? (Hint: chapter 6, histogram buckets.)

✅ **You learned:** each SLI catches a different kind of regression.

Lab 5: Break the monitoring (15 min)

```
scripts/chaos-prometheus-outage.sh
```

Read the output. The AnalysisRun phase is `Error` (not `Failed`), with `consecutiveError=5`.

✅ **You learned:** fail-safe design. Not being able to measure is treated as "not safe to promote".

Lab 6: Prove the network isolation (5 min)

```
scripts/check-network-policy.sh
```

Then try it by hand from another namespace; it should time out:

```
kubectl create ns test-outside
kubectl -n test-outside run probe --rm -it --image=busybox:1.37 --restart=Never -- \
  wget -T 4 -q0- http://orders-api.delivery.svc.cluster.local/version
kubectl delete ns test-outside
```

✅ **You learned:** NetworkPolicy is a firewall between Pods, and it really is enforced.

Lab 7: Change the SLO and see the gate move (15 min)

1. In `charts/orders-api/values.yaml`, set `slo.availabilityTarget: 0.999`.
2. `helm template orders-api charts/orders-api | grep successCondition`. The limit is now `0.0144`.
3. Release with a **smaller** fault: `bash helm upgrade orders-api charts/orders-api -n delivery --reuse-values -f charts/orders-api/values.yaml \ --set image.tag=1.9.0 --set-string fault.errorRate=0.03` (Build the image first: `scripts/build-image.sh 1.9.0`.)
4. **Expect an abort.** With the old 99.5% SLO, 3% errors would have passed (limit 7.2%). Revert the file afterwards.

✅ **You learned:** the business target (the SLO) directly controls release safety, and Helm computes the technical limit from it.

Lab 8: Test the alerts (15 min)

```
make slo-test
```

Open `slo/tests/orders-api-slo.test.yaml` and change the healthy test from 0.1% errors to 5% errors (values: `'0+95x150'` and `'0+5x150'`). Run it again: it **fails**, because now alerts *do* fire. Read the failure message, then revert.

✅ **You learned:** alert logic can be unit-tested like code.

Stretch goals

- **Change the steps:** set `canary.weights: [10, 30, 50, 100]` with `replicas: 10` and observe the finer steps.
- **Add a metric:** add a third analysis metric (for example "the canary must receive at least 1 req/s") to `analysistemplate.yaml`.
- **Fill the alerts panel:** add Alertmanager and route the `page` alerts to a webhook.

When you are done: `make down`.

Glossary

Term	Meaning
Abort (rollout)	Argo Rollouts stops a release and returns all traffic to the stable version
AnalysisRun	One execution of an AnalysisTemplate during a release, with stored measurements and a phase (Running / Successful / Failed / Error)
AnalysisTemplate	Reusable definition of the metrics and success conditions used to judge a release
Annotation	Key/value metadata on a Kubernetes object, read by tools (for example <code>prometheus.io/scrape: "true"</code>)
Burn rate	Current error ratio ÷ error budget. 1x lasts exactly the SLO window; 14.4x spends 2% of a 30-day budget per hour
Canary release	Sending a small, growing share of traffic to a new version while checking its health
CD	Continuous Delivery/Deployment: automatically releasing changes that passed CI
CI	Continuous Integration: automatically building and testing every change
ClusterRole / RoleBinding	RBAC objects that define and grant permissions
ConfigMap	Kubernetes object holding non-secret configuration files
Container / image	A running isolated process / the immutable package it starts from
Controller	A program that watches Kubernetes objects and works to make reality match the desired state
Counter	Metric type that only increases (for example total requests)
CRD	Custom Resource Definition: adds a new object type to Kubernetes (for example Rollout)
Error budget	1 – SLO: the share of requests allowed to fail (0.5% for a 99.5% SLO)
Fail safe	Design where a failure of the safety system leads to the safe outcome (here: abort, not promote)
Gauge	Metric type that can go up and down
Helm chart / release	Templated Kubernetes package / one installed instance with revision history
Histogram	Metric type that counts observations into buckets, used for latency percentiles
kind	Kubernetes IN Docker: a real cluster whose nodes are containers
Label / selector	Key/value tag on objects / a query that matches objects by their labels
Liveness / readiness probe	Health check that restarts a container / that adds or removes it from traffic
Multi-window alert	Alert that requires a long AND a short window to exceed a threshold
Namespace	A grouping and isolation boundary inside a cluster
NetworkPolicy	Firewall rules between Pods
p95 latency	95% of requests are faster than this value

Term	Meaning
Pod	The smallest deployable unit in Kubernetes: one or more containers
Pod-template hash	Label (rollouts-pod-template-hash) identifying which version a Pod belongs to
Progressive delivery	Releasing gradually while verifying health at each step
PromQL	Prometheus' query language
Recording rule	Pre-computed, named PromQL result stored as a new time series
Reconciliation	The loop that keeps changing the cluster until it matches the declared state
Relabelling	Prometheus rules that add, change or drop labels during discovery or scraping
ReplicaSet	Keeps a set number of identical Pods running
Rollback	Returning to a previous version
Rollout	Argo Rollouts object replacing a Deployment, with canary or blue/green strategies
Scrape	Prometheus fetching /metrics from a target
Server-side apply	Kubernetes apply mode that tracks which tool owns each field (default in Helm 4)
Service	Stable network name and load balancer in front of matching Pods
SLA	Service Level Agreement: an external contract, usually with penalties
SLI	Service Level Indicator: a measured ratio of good events (for example non-5xx requests ÷ all requests)
SLO	Service Level Objective: the target for an SLI over a window (for example 99.5% over 30 days)
Stable version	The version currently trusted to serve all traffic
Time series	One metric name + one unique label set, with values over time
Trivy	Scanner for vulnerabilities in images, filesystems and configuration

Interview questions this project prepares you for

Try to answer each question out loud first, then open the answer.

SRE and SLOs

1. What is the difference between an SLI, an SLO and an SLA?

▼ Answer

An SLI is a measurement (non-5xx requests ÷ all requests). An SLO is the internal target for it (99.5% over 30 days). An SLA is an external contract with consequences, usually looser than the SLO.

2. What is an error budget and how do teams use it?

▼ Answer

1 – SLO: the amount of failure allowed. With budget left, teams ship faster; when it runs out, they prioritise reliability work. It turns reliability into a shared, measurable trade-off.

3. Explain burn rate. What does 14.4x mean?

▼ Answer

Error ratio ÷ budget. 14.4x spends the 30-day budget in about 2 days, which is 2% of the budget per hour. It is the classic fast-burn paging threshold.

4. Why use multi-window, multi-burn-rate alerts instead of "errors > X% for 5 min"?

▼ Answer

The long window ensures significance (no paging on blips). The short window ensures the problem is current, so alerts resolve quickly after a fix. Different burn rates separate "page now" from "fix this week".

5. How did you test that your alerts actually fire?

▼ Answer

With `promtool test rules` : synthetic series for 10%, 4%, 0.1% errors and slow requests, asserting which alerts fire at which time, and that none fire when healthy.

Progressive delivery

6. Compare rolling update, blue/green and canary.

▼ Answer

Rolling update replaces Pods gradually but doesn't check health. Blue/green switches all traffic at once between two full environments (fast rollback, but all-or-nothing). Canary shifts a growing share while measuring, so it limits the blast radius.

7. How does your canary decide to promote or roll back?

▼ Answer

A background AnalysisRun queries Prometheus every 30 s for the canary pods only (by pod-template hash): error ratio ≤ 0.072 and p95 ≤ 300 ms. Two failed measurements abort the rollout; passing all steps promotes it.

8. Why is the canary threshold 0.072?

▼ Answer

Budget $0.5\% \times \text{max burn rate } 14.4 = 7.2\%$. It's the same as the fast-burn page, so a version that would page at full traffic is never promoted. It's computed in Helm from the SLO, not hard-coded.

9. How do you make sure the analysis measures only the new version?

▼ Answer

Prometheus relabelling copies the `rollouts-pod-template-hash` Pod label onto every sample. The Rollout passes the canary's hash to the AnalysisTemplate, and every query filters on it.

10. What happens if Prometheus is down during a release?

▼ Answer

Queries error out. After `consecutiveErrorLimit` (4) is exceeded, the AnalysisRun ends in `Error` and the rollout aborts. That's fail safe: unverified releases are never promoted. It was tested with a chaos script (aborted 60 s after the outage).

11. What if the canary receives no traffic?

▼ Answer

The query returns no data, `len(result) > 0` fails, and the release isn't promoted. "No evidence" is treated as "not verified".

12. Your canary weights are 20/40/60/80. How is traffic split without a service mesh?

▼ Answer

By replica ratio: one Service selects all Pods, so 1 of 5 Pods gets about 20%. The trade-off is granularity of 1/replicas. A traffic router (Gateway API, Istio, NGINX) gives exact percentages.

13. What regression would slip through your canary, and how is it caught?

▼ Answer

One below the gate, for example 4% errors. The 6x slow-burn alert (6h and 30m windows) catches it, and that is proven in the promtool tests.

Kubernetes and Helm

14. Readiness vs liveness probe?

▼ Answer

Readiness controls whether the Pod receives traffic. Liveness controls whether the container gets restarted.

15. How does your service shut down gracefully?

▼ Answer

On SIGTERM it fails `/readyz` so it is removed from the Service, waits 5 s for in-flight requests, then stops. `terminationGracePeriodSeconds` is 20.

16. Which pod security controls did you apply?

▼ Answer

Non-root UID, read-only root filesystem, no privilege escalation, all capabilities dropped, seccomp RuntimeDefault, no service-account token, CPU and memory limits, and a NetworkPolicy.

17. What was the Helm 4 server-side-apply problem?

▼ Answer

Helm 4 owns every field it applies. Argo Rollouts rewrites the selectors of stable/canary Services, so the next upgrade conflicted on `.spec.selector`. I removed those Services and scoped the analysis by pod hash, so every field has one owner.

18. Why template the threshold in Helm instead of writing 0.072?

▼ Answer

One source of truth: change the SLO and the gate follows automatically. It avoids drift between the business target and the technical limit.

Observability

19. Why `rate()` on counters, and why `or vector(0)` in the error ratio?

▼ Answer

`rate()` gives the per-second increase and handles counter resets. or `vector(0)` makes the numerator 0 when no 5xx series exists yet, instead of an empty result.

20. How is p95 computed, and what limits its accuracy?

▼ Answer

`histogram_quantile(0.95, ...)` over bucket rates. It interpolates inside buckets, so precision depends on the bucket boundaries (600 ms read as 0.975 s within the 0.5–1 s bucket).

21. Why are recording rules used?

▼ Answer

They pre-compute expensive queries over long windows, give them stable names that alerts and dashboards share, and make evaluation cheap.

CI/CD and DevSecOps

22. What does your pipeline test, and in what order?

▼ Answer

The fast checks run in parallel: unit tests, lint, shellcheck, promtool, helm lint, kubeconform, and the image build plus Trivy. Only if all pass does a kind cluster run the real release scenarios, the chaos test and the network-policy test. Diagnostics are uploaded on failure.

23. The image scan failed. What did you do?

▼ Answer

The findings came from pip's vendored libraries, setuptools and an OS package, not the app. I removed pip and setuptools from the runtime image and applied OS updates, which got the scan to 0 findings. I fixed the cause rather than suppressing it.

24. Why pin tool versions in CI?

▼ Answer

Reproducibility: the same commit gives the same result, and upstream changes can't silently break or weaken the pipeline.

25. What would you add for production?

▼ Answer

A traffic router for fine-grained weights, Alertmanager routing, GitOps with Argo CD, persistent and long-term metrics storage (Thanos or Mimir) for real 30-day windows, and signed images with admission

verification.